

Programming assignment

2025 Fall - 機器學習

111652001 吳文生

1. (Classification using GDA) Your task is to use Gaussian Discriminant Analysis (GDA) to build a classification model. To complete this assignment, make sure you:
 - (a). Write your own code to implement the GDA algorithm. (**Do not use built-in classification functions.**)
 - (b). Clearly explain how the GDA model works and why it can be used for classification, in particular this data set.
 - (c). Train your model on the given dataset and report its accuracy. Be explicit about how you measure performance (e.g., accuracy on a test set, cross-validation, etc.).
 - (d). Plot the decision boundary of your model and include the visualization in your report.

Our code(*Programming_assignment_w4_1.py*) provided implements a **Gaussian Discriminant Analysis (GDA)** classifier from scratch. It handles data parsing from the O-A0038-003.xml file, prepares a dataset for binary classification, trains the custom GDA model, and evaluates its performance. It also visualizes the model's decision boundary.

Gaussian Discriminant Analysis (GDA) is a **generative learning algorithm** that models the probability distributions of data for each class, rather than directly learning a decision boundary like a discriminative model (e.g., logistic regression). GDA models the conditional probability of the input features given the class, $p(x|y)$, and the prior probability of the class, $p(y)$.

The core of GDA relies on a few key assumptions:

- **Gaussian Assumption:** It assumes that the data within each class is normally distributed.

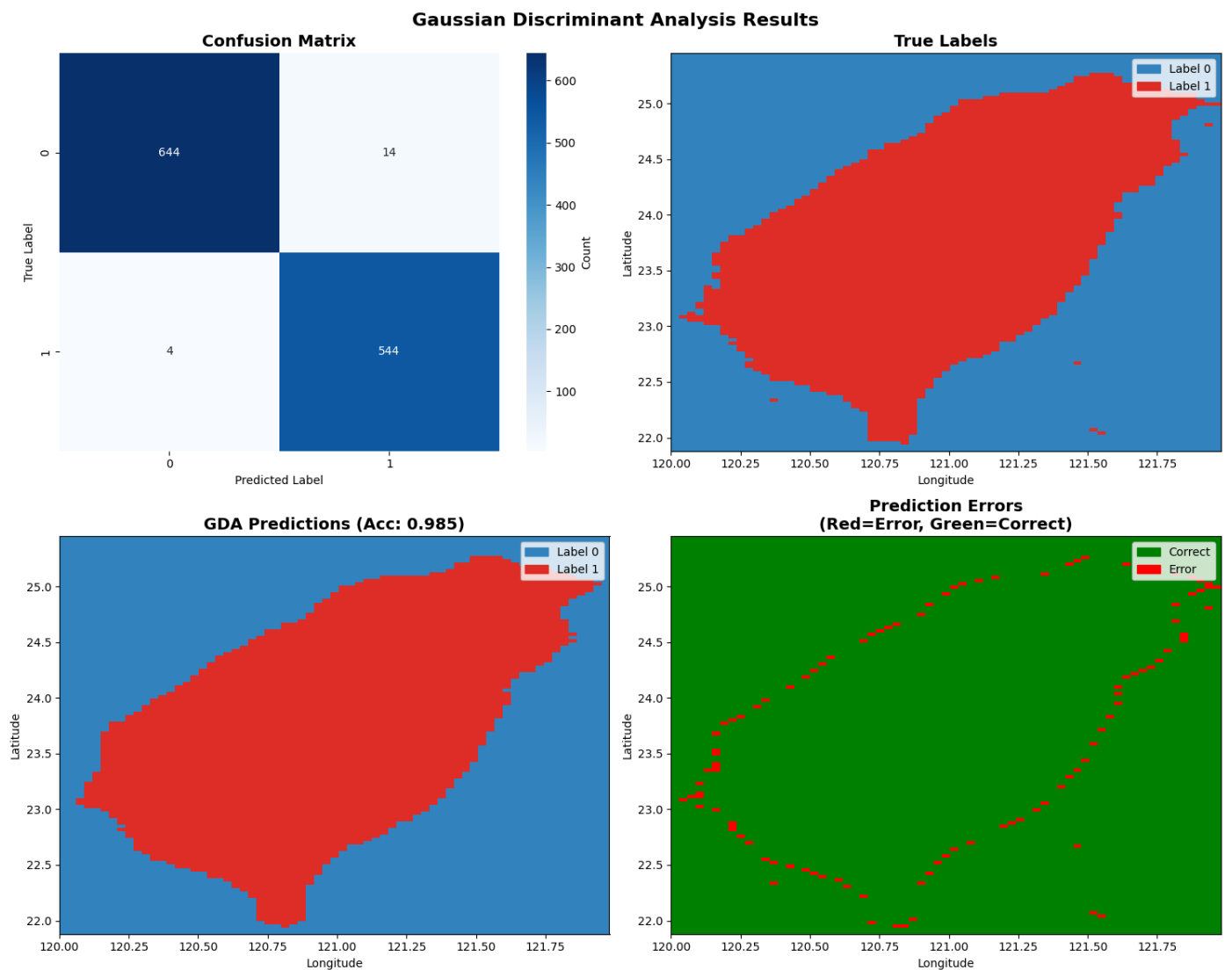
- **Shared Covariance:** Both classes share the same covariance matrix, Σ .

To classify a new data point, GDA uses **Bayes' rule** to calculate the posterior probability,

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}.$$

It then predicts the class with the highest posterior probability.

The performance metric we used is **accuracy**, which calculates the percentage of correct predictions the model makes on the validation and test sets. The final accuracy reported on the test set was 98.51%.



2. (Regression) Your task is to build a regression model that represents a piecewise smooth function. To do this, combine the two models from Assignment 4 into a single function. Specifically, let

- $C(x)$ be your classification model, and
- $R(x)$ be your regression model.

Then construct a model $h(x)$ defined as

$$h(x) = \begin{cases} R(x), & \text{if } C(x) = 1 \\ -999, & \text{if } C(x) = 0 \end{cases}$$

To complete this assignment, make sure you:

- (a) Implement this combined model in code.
- (b) Apply your model to the dataset and verify that the piecewise definition works as expected.
- (c) Briefly explain how you built the combined function.
- (d) Include plots or tables that demonstrate the behavior of your model.

The combined piecewise function $h(x)$ integrates two independent models:

1. Classification Model $C(x)$: Random Forest Classifier

- Trained on all data points (longitude, latitude) with binary labels
- Label 0: Invalid data (-999), Label 1: Valid temperature data
- Purpose: Identify which locations have valid measurements

2. Regression Model $R(x)$: Random Forest Regressor

- Trained only on valid data points (where label = 1)
- Learns the relationship between geographic coordinates and temperature
- Purpose: Predict temperature values for valid locations

3. Piecewise Function $h(x)$:

For any input (longitude, latitude):

- First, apply $C(x)$ to classify if the location is valid
- If $C(x)=1$ (valid): return temperature prediction from $R(x)$
- If $C(x)=0$ (invalid): return -999

